

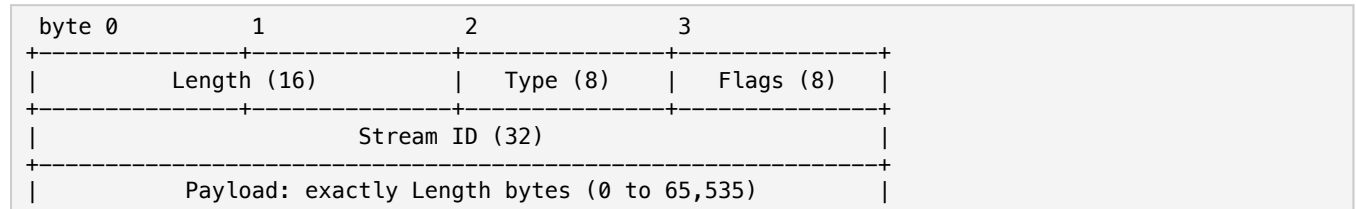
WireHTTP/1 — HTTP in binary frames

Network Architecture course project · protocol spec, version 1 · Author: Shambhu Yadav (24bcs10356)

Scope. WireHTTP/1 carries HTTP requests and responses over one TCP connection. The *meaning* is unchanged HTTP: methods, status codes and header names keep their HTTP meanings. Only the *encoding* on the wire is new. MUST, SHOULD and MAY are used as in the RFCs. **Every multi-byte integer is big-endian (network byte order).**

1. Frames

Everything sent on the connection, in both directions, is a sequence of frames. A frame is an 8-byte header followed by exactly Length bytes of payload.



Field	Bits	Meaning
Length	16	Payload size in bytes, not counting the 8-byte header. Every value 0–65,535 is legal and receivers MUST accept all of them.
Type	8	0x00 DATA (body bytes), 0x01 HEADERS (a header block, §3). Any other value: see §2.
Flags	8	0x01 END_STREAM: the last frame of this request or response. Other bits: senders set them to 0, receivers MUST ignore them.
Stream ID	32	The request this frame belongs to. Requests use 1, 2, 3, ...; 0 means the connection itself and never carries a request.

2. Unknown frame types — room for version 2

A receiver (client or server) that meets a frame type it does not know **MUST read and discard exactly Length payload bytes, then carry on with the next frame. It **MUST NOT** treat this as an error or close the connection.** The length is always in the same place, so an old receiver can always step over a frame type invented later.

3. Header block (payload of a HEADERS frame)

Fields follow each other until the payload ends: Index (8) | [NameLen (8) | Name] | ValueLen (16) | Value

- **Index 1–10:** the name is taken from the table below, costing one byte instead of the text.
- **Index 0:** a literal name follows: NameLen (1–255), then that many bytes of lower-case ASCII. Used for any name not in the table.
- **Index 11–255:** reserved for v2. A receiver **MUST** skip the field (its value is still length-prefixed) and carry on.
- **Values** are length-prefixed byte strings, normally ASCII text, *including* :status ("404") and content-length ("15"). No compression of any kind.
- A field that would run past the end of the payload, or a NameLen of 0, makes the block **malformed**.

Index	Name	Sent by	Index	Name	Sent by
1	:method	client	6	:status	server
2	:path	client	7	content-type	server
3	:authority	client	8	content-length	server
4	user-agent	client	9	server	server
5	accept	client	10	date	server

4. Requests, responses and the connection

Request. Exactly one HEADERS frame with END_STREAM set (v1 requests have no body). It **MUST** contain exactly one :method and exactly one :path, and **SHOULD** contain :authority. The client numbers its requests 1, 2, 3, ... on each connection. Version 1 defines only GET.

Response. Sent on the request's stream ID: one HEADERS frame (END_STREAM clear) containing :status, then one or more DATA frames, the last of which has END_STREAM set. An empty body is a single DATA frame with Length 0 and END_STREAM. **The frames decide where the body ends.** If content-length is sent it **MUST** equal

the total of the DATA lengths, and a client that sees a mismatch **MUST** treat the response as broken. Error responses carry a short text body like any other response.

One at a time. A client **MUST NOT** send its next request until the previous response has ended (END_STREAM). A HEADERS or DATA frame for any stream other than the open request is a protocol error, and the client **MAY** close the connection.

Status	When
200	A regular file exists at :path under the server's root (/ means /index.html). The body is its bytes.
400	Malformed: HEADERS on stream 0 · request HEADERS without END_STREAM · DATA sent by a client · malformed header block · :method or :path missing or repeated · :path not starting with /, containing a .. segment, or containing a NUL byte.
404	Nothing (or not a regular file) at that path under the root.
405	:method is not GET.

Connection lifetime. The server **MUST** keep the connection open after **every** response, including 400, 404 and 405. Length is always at the same position, so the server always knows where the next frame starts and a bad request cannot put the two sides out of step. Either side ends the connection by closing TCP.

5. Why these widths (HTTP/2 chose 24 / 8 / 8 / 31)

Field	HTTP/2	Ours	Reason
Length	24	16	v1 has no SETTINGS frame to agree a maximum, so the spec fixes one. With 16 bits every value is legal: there is no "frame too big" error, each receiver needs one fixed 64 KiB buffer, and a peer cannot make it allocate megabytes. Long bodies become several DATA frames closed by END_STREAM (length-prefixed chunks plus an end marker, like chunked encoding); with 16 KiB frames the header costs 0.05 %.
Type	8	8	2 types used, 254 left for v2, made safe by §2.
Flags	8	8	1 bit used (END_STREAM), 7 left; unknown bits are ignored.
Stream ID	1 + 31	32	Every frame names its request, so responses never have to be matched by arrival order (HTTP/1.1's weakness), and v2 can pipeline or multiplex without a new header. There is no odd/even split because our server never starts streams (no push), and no reserved bit because Type and Flags already leave room.
Total	9 B	8 B	Length comes first, so a reader knows how far to skip before interpreting anything else. Every field is a whole number of bytes: read it with ntohs/ntohl, with no 24-bit or 31-bit masking.

Why HTTP/2 chose 24/8/8/31. HTTP/2 multiplexes many streams on one TCP connection, so frames from different streams interleave and are kept small (default maximum 16,384 bytes, raised only by agreement in SETTINGS). 24 bits covers any size the peers may agree on and is one byte cheaper than 32 on every frame. One byte of type covers its ten frame types; one byte of flags covers END_STREAM, END_HEADERS and the rest. The stream ID is 31 bits plus a reserved bit. Clients use odd IDs and servers use even IDs, so both sides could open streams (server push) without coordinating. Stream 0 is the connection itself (SETTINGS, PING, GOAWAY).

6. Worked example: GET /index.html from localhost:9000 (61 bytes)

00 35 01 01 00 00 00 01	Length 0x35 = 53 · Type 01 HEADERS · Flags 01 END_STREAM · Stream 1
01 00 03 47 45 54	[1] :method 3 "GET"
02 00 0b 2f 69 6e 64 65 78 2e 68 74 6d 6c	[2] :path 11 "/index.html"
03 00 0e 6c 6f 63 61 6c 68 6f 73 74 3a 39 30 30 30	[3] :authority 14 "localhost:9000"
04 00 07 77 63 75 72 6c 2f 31	[4] user-agent 7 "wcurl/1"
05 00 03 2a 2f 2a	[5] accept 3 "*/*"

The same request as HTTP/1.1 text is 84 bytes. HEXDUMP.md annotates a complete request and response byte by byte.